

Assignment: Functions in C++

Question 1 – Function Prototype and Return Value

Write a C++ program that calculates the **area of a circle**.

- Place a **function prototype** `double circleArea(double radius);` **before** `main()`.
- Define the function **after** `main()`.
- The function must receive the radius **by value** and **return** the computed area.
- In `main()`, prompt the user for the radius, call the function, and display the result.
- Use `const double PI = 3.14159;` either as a global constant or locally inside the function.

Question 2 – Passing Data by Value

Write a program that demonstrates that **passing arguments by value does not modify the original variables**.

- Create a function `void tryToModify(int num)` that multiplies its parameter by 2 and prints the new value inside the function.
- In `main()`, declare an integer variable, initialise it with a user-supplied value, print it, call `tryToModify()`, and print the variable again.
- Add comments explaining why the original variable remains unchanged after the function call.

Question 3 – Returning a Boolean Value

Write a program containing a function `bool isPrime(int n)` that **returns** `true` if `n` is a prime number and `false` otherwise.

- The function must accept its argument **by value** and **return a boolean**.
- Inside `main()`, ask the user for an integer, call the function, and print "Prime" or "Not Prime" based on the returned value.
- Include a **function prototype** for `isPrime()` before `main()`.

Question 4 – Local and Global Variables

Write a program that highlights the difference between **local and global** variables.

- Declare a **global** integer variable `counter` initialised to 0.
- Write a function `void updateCounter()` that:
 - Declares a **local** integer variable `localCounter` initialised to 0 (do **not** use `static`).
 - Increments **both** `counter` (global) and `localCounter`.
 - Prints the current values of `counter` and `localCounter`.
- In `main()`, call `updateCounter()` three times.
- Add a comment in your code explaining why `localCounter` is always 1 while `counter` increases with each call.

Question 5 – Menu-Driven Program Using Functions

Design a **menu-driven program** that performs temperature conversions.

- Display a menu repeatedly until the user chooses to quit:
 1. Fahrenheit to Celsius
 2. Celsius to Fahrenheit
 3. QuitEnter your choice:
- Implement two functions:
 - `double fahrenheitToCelsius(double f)` – takes Fahrenheit **by value** and **returns** Celsius.
 - `double celsiusToFahrenheit(double c)` – takes Celsius **by value** and **returns** Fahrenheit.
- Place **function prototypes** above `main()` and define the functions below `main()`.
- For choices 1 and 2, prompt the user for the temperature, call the appropriate function, and display the result.
- Use the `return` statement in both functions; do **not** print inside the conversion functions.
- Validate the menu choice (re-prompt on invalid input).